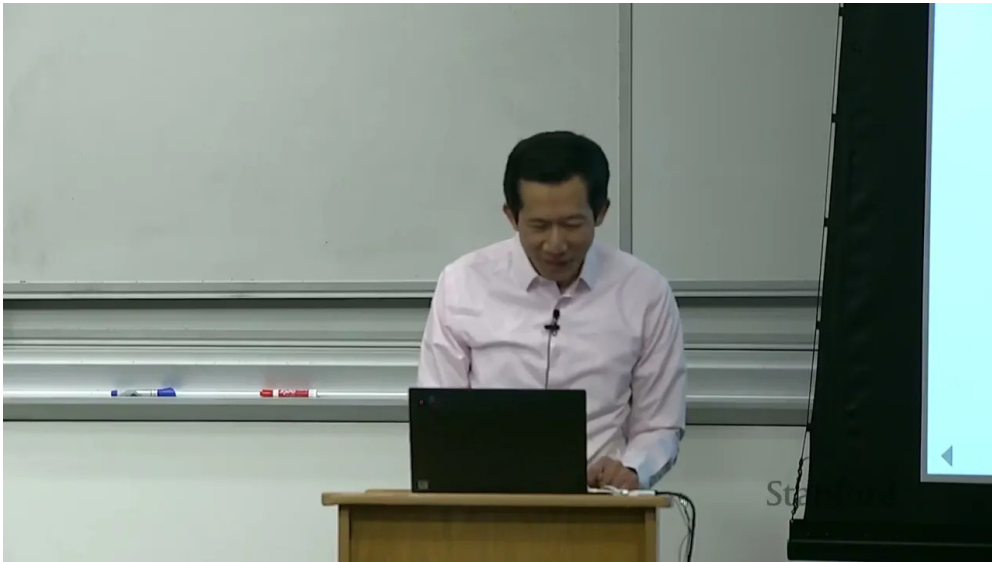


课程笔记

CS336 Lecture 8: Distributed Training Across Multiple GPUs

基于课程字幕与代码脚本整理

基于公开视频字幕整理



视频作者/频道: Stanford CS336

发布日期: 未提供

视频时长: 约 1:15:00

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 导言：多 GPU 训练到底在和什么作战	3
2 集合通信：分布式编程的基本积木	4
2.1 术语和直觉	4
2.2 常见原语	4
2.3 为什么这些原语重要	5
3 硬件与软件栈	5
3.1 旧式路径和现代路径	5
3.2 互联带宽对比：NVLink vs PCIe vs InfiniBand	6
3.3 NCCL 与 <code>torch.distributed</code>	7
3.4 带宽基准的意义	7
3.5 通信复杂度怎么估	7
3.6 all-reduce 和 reduce-scatter 的关系	8
3.7 怎么读一个 all-reduce 的数字	9
4 数据并行：按 batch 切分	10
4.1 工作方式	10
4.2 DDP 的代价与边界	11
4.3 为什么 DDP 不是“把单卡训练复制四份”就结束	11
4.4 内存账本：DDP 到底在占什么	11
4.5 一个 DDP 的数值化判断	13
4.6 Worked Example: Llama 2 13B 的 DDP 通信开销	13
5 张量并行：按宽度切分	14
5.1 工作方式	14
5.2 张量并行 vs 数据并行：通信量对比	15
5.3 为什么张量并行的反向传播更难	16
5.4 一个张量并行的数值化例子	16
6 流水线并行：按深度切分	17
6.1 为什么要 microbatch	17
6.2 流水线为什么总有空档	17
6.3 点对点通信	18
6.4 流水线效率的近似公式	19
6.5 Worked Example: 流水线 bubble 开销的精确计算	19
6.6 stage 负载不均会怎样	20
7 如何选择并行策略	21
7.1 三种策略的直观对比	21
7.2 三种并行策略的详细对比	21
7.3 一个实用的决策表	22

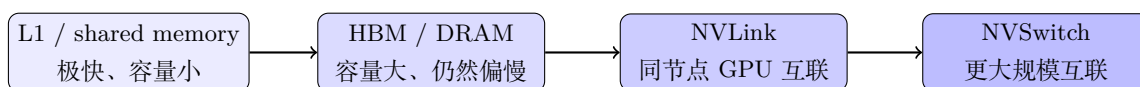
8 一个四卡算例：同一模型的三种切法	23
9 实际系统中的并行策略	24
9.1 ZeRO：在数据并行中消除冗余	24
9.2 实际训练框架的并行策略对比	25
10 全讲总结	25
10.1 本章小结	25
10.2 这节课没有覆盖的内容	26
10.3 拓展阅读	26

1 导言：多 GPU 训练到底在和什么作战

这节课的主题不是“多开几张卡”，而是更根本的问题：当模型、batch、状态和激活都在膨胀时，如何把训练系统重新切成多个设备可以协同执行的形状。讲者一开始就把核心矛盾说得很直白：计算离数据总是有距离，而性能优化的关键，就是尽量减少数据搬运，把大部分工作放在离数据最近的地方完成。¹

本讲的统一主题

1. 上一讲在单 GPU 内通过 fusion 和 tiling 减少 HBM 往返。
2. 这一讲在多 GPU / 多节点内通过 replication、sharding 和 collective 减少跨设备搬运。
3. 两讲本质相同：都在想办法提高算术强度，避免让搬数据压倒算数据。



越靠近算子执行位置，越快；越依赖主机中转，越慢。

图 1: 从单机层次到跨设备层次：多 GPU 训练只是把内存层次扩展到了更远的通信层次

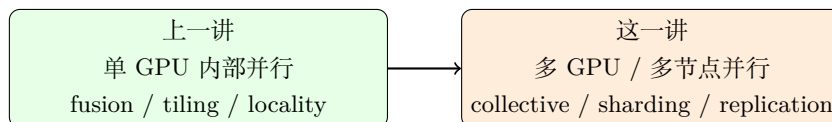


图 2: 从单卡局部优化过渡到跨设备全局优化

一句话记忆

如果数据已经在本地缓存里，计算就快；如果数据要穿过别的 GPU 或别的节点，通信就会成为瓶颈。分布式训练的核心，不是“多加机器”，而是“让每次通信都物有所值”。

层次	典型位置	主要矛盾
L1 / shared memory	单 GPU 内部	容量小但速度快
HBM / DRAM	单 GPU 内部	容量大但比片上慢
NVLink / NVSwitch	多 GPU / 多节点	带宽高，但仍然稀缺
Ethernet / 主机中转	跨节点	延迟和带宽都不理想

表 1: 分布式系统里的层次矛盾：越远越慢，越慢越要避免频繁访问

¹根据字幕 00:00:05-00:02:04，讲者用“上一讲单 GPU 内并行”过渡到“这一讲多 GPU / 多节点并行”，并明确强调避免 data transfer bottlenecks。

2 集合通信：分布式编程的基本积木

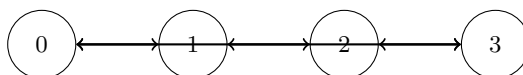
在分布式训练里，最小的抽象不是“GPU”，而是 **collective operations**。它们描述的是一组 rank 之间的标准通信模式，比手工点对点通信更清晰，也更容易被 NCCL 和 PyTorch 优化。

2.1 术语和直觉

- **World size**: 参与通信的设备数量。
- **Rank**: 某一台设备的编号，通常从 0 开始。
- **Collective**: 所有 rank 必须配合完成的一次通信动作。

为什么“collective”这个词重要

它强调的不是“发一条消息”，而是“大家一起完成一个固定协议”。这也是为什么分布式程序最怕漏掉一个 rank：不是少传了一条消息，而是整套协议卡住了。



collective 是全局协议：少一个 rank，大家都会等。

图 3: collective 不是局部动作，而是全局协议

2.2 常见原语

原语	直觉	用途
broadcast	一份数据发给所有 rank	参数同步、配置下发
scatter	一份输入切成多份分发	数据切片
gather	多份数据收集到一个 rank	汇总输出
reduce	对多个值做 sum/min/max 等归约	梯度聚合、统计合并
all-gather	每个 rank 都拿到完整拼接结果	张量并行中的激活恢复
reduce-scatter	先归约，再切片分发	all-reduce 的组成部分
all-reduce	所有 rank 最终都拿到归约结果	DDP 梯度同步

表 2: 常见集合通信原语

三个容易混淆的点

1. **reduce** 不是任意函数，而通常要求可结合、可交换，例如 sum、min、max。
2. **all** 的意思是“所有 rank 都得到结果”。
3. **all-reduce** 常被实现为 **reduce-scatter** + **all-gather**，这也是很多底层优化的切入点。

原语	输入形状	输出形状	每 rank 发送量	典型用途
broadcast	rank 0 持有 S	所有 rank 持有 S	S (root 发出)	初始化参数同步
scatter	rank 0 持有 $p \times S$	每个 rank 持有 S	S (root 发出)	数据切片分发
gather	每个 rank 持有 S	rank 0 持有 $p \times S$	S (每 rank 发出)	汇总输出、收集指标
reduce	每个 rank 持有 S	rank 0 持有 S	S (每 rank 发出)	梯度求和到 root
all-gather	每个 rank 持有 S	所有 rank 持有 $p \times S$	$\frac{p-1}{p} \cdot pS$	张量并行中恢复完整激活
reduce-scatter	每个 rank 持有 pS	每个 rank 持有 S	$\frac{p-1}{p} \cdot pS$	all-reduce 的前半段
all-reduce	每个 rank 持有 S	所有 rank 持有 S	$2 \cdot \frac{p-1}{p} \cdot S$	DDP 梯度同步 (最常用)

表 3: 集合操作完整总结: S 为单份数据大小 (bytes), p 为 rank 数。通信量指 ring 实现下每个 rank 的发送量。

如何记忆通信量公式

对于 ring 实现, 核心因子是 $\frac{p-1}{p}$: 每个 rank 在 $p-1$ 步中, 每步发送 $\frac{1}{p}$ 的数据。当 p 很大时, $\frac{p-1}{p} \approx 1$, 也就是说每个 rank 几乎要把自己持有的全部数据发一遍。all-reduce 之所以有系数 2, 是因为它包含 reduce-scatter 和 all-gather 两步, 每步各传一次。

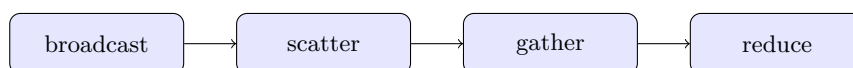


图 4: 广播、切分、收集与归约是最基本的通信语义

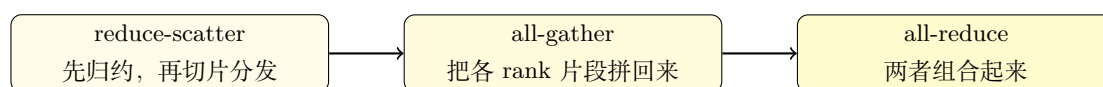


图 5: all-reduce 的常见拆法: reduce-scatter 加 all-gather

2.3 为什么这些原语重要

collective operations 不是“附加工具”, 而是整个分布式训练的语言。你先决定模型或数据如何切分, 再决定哪些切片必须通过 collective 对齐。训练策略的区别, 本质上就是 **谁保留本地、谁需要交换、什么时候交换**。

同步点不是可选项

如果某个 rank 少做了一次 collective, 其他 rank 就会一直等。训练代码里最容易出现 bug 之一, 就是某个分支上漏掉了一个同步点, 程序表面上看像“挂住了”, 其实是协议不完整。

3 硬件与软件栈

3.1 旧式路径和现代路径

早期的常见路径是 GPU 经过 PCIe 到 CPU, 再经过主机网卡和 Ethernet 去和别的节点通信。这个链路能工作, 但太长、太慢、太容易被主机中转拖累。现代 data center 倾向于让 GPU 之间尽量直连, 用 NVLink 或 NVSwitch 绕开主机路径。

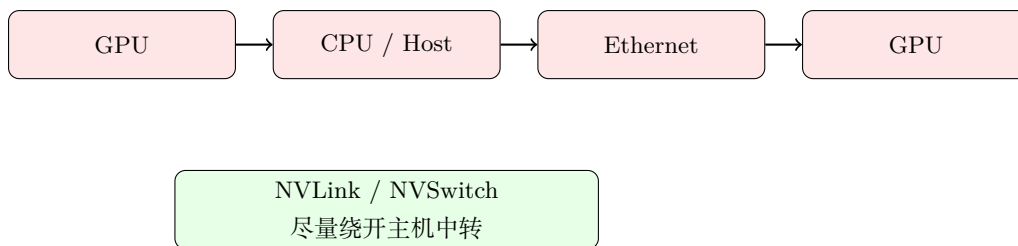


图 6: 通信路径的演化：越依赖主机中转，越容易被搬运成本拖慢

一个现实约束

PCIe、Ethernet、NVLink、NVSwitch 的性能会变，但层次关系不会变：**越接近算子执行位置，越快；越依赖主机中转，越慢。**

3.2 互联带宽对比：NVLink vs PCIe vs InfiniBand

选择并行策略时，硬件互联的带宽和延迟是最关键的约束之一。下表汇总了当前主流的 GPU 间互联技术：

互联技术	单向带宽	典型延迟	作用范围	工程影响
NVLink 4.0 (H100)	900 GB/s (双向)	$\sim 1\text{--}2\ \mu\text{s}$	同节点 GPU	张量并行的理想链路
NVSwitch (DGX)	900 GB/s (全互联)	$\sim 1\text{--}5\ \mu\text{s}$	同节点 8 GPU	8 卡全互联，无需 ring
PCIe 5.0 x16	64 GB/s (双向)	$\sim 2\text{--}5\ \mu\text{s}$	GPU-CPU	跨 CPU 的 GPU 通信瓶颈
InfiniBand NDR	400 Gb/s ≈ 50 GB/s	$\sim 1\text{--}2\ \mu\text{s}$	跨节点	跨节点数据并行的主力
RoCE / Ethernet 100G	100 Gb/s ≈ 12.5 GB/s	$\sim 5\text{--}20\ \mu\text{s}$	跨节点	低端集群的折衷方案

表 4: 主流 GPU 互联技术带宽对比（数值为近似峰值，实际有效带宽通常为峰值的 60%–85%）

带宽差异如何影响并行策略选择

- **NVLink 带宽是 InfiniBand 的 ~ 18 倍**：张量并行每层都要通信，必须放在 NVLink 可达的范围内（同一节点）。跨节点做张量并行几乎不可行。
- **InfiniBand 带宽是 Ethernet 的 ~ 4 倍**：数据并行的梯度同步每步只发生一次，用 InfiniBand 跨节点通常可以接受；但用普通 Ethernet 就会明显拖慢训练。
- **PCIe 是隐藏瓶颈**：即使同一节点，如果 GPU 之间没有 NVLink 而只能走 PCIe $\rightarrow$ CPU $\rightarrow$ PCIe，带宽会从 900 GB/s 骤降到 64 GB/s。

一个实际的带宽换算

H100 的 NVLink 双向总带宽 900 GB/s。如果要做一次 all-reduce 同步 26 GB (Llama 2 13B 的梯度)，在 ring 实现下每 rank 传输约 $2 \times \frac{p-1}{p} \times 26 \approx 39\text{ GB}$ (8 卡)。用 NVLink 大约 $39/450 \approx 87\text{ ms}$ (取单向有效带宽 450 GB/s)；用 InfiniBand 则需要 $39/40 \approx 975\text{ ms}$ 。这就是为什么跨节点做 all-reduce 要谨慎。

3.3 NCCL 与 torch.distributed

NVIDIA 的 NCCL 负责把高层 collective 翻译成低层通信包和 CUDA kernel。它会先识别硬件拓扑，再决定走哪条链路、哪种调度方式、哪些传输可以并行进行。

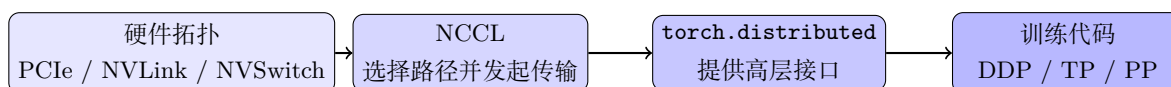


图 7: 从硬件拓扑到训练代码: NCCL 和 PyTorch 分别负责低层调度与高层接口

在 PyTorch 里, `torch.distributed` 提供了更高层的封装:

- `init_process_group` 初始化分布式环境;
- `all_reduce`、`reduce_scatter_tensor`、`all_gather_into_tensor` 对应 collective;
- `send / recv` 用于点对点通信;
- `barrier` 用于显式同步;
- `destroy_process_group` 用于清理。

分布式编程里的同步点

`all_reduce` 这类函数本身就是同步点。少一个 rank, 大家都会等。很多分布式 bug 不是算错, 而是某个进程没有走到同一个协议位置。

3.4 带宽基准的意义

讲里专门演示了如何通过脚本测量 NCCL 带宽。这个动作的意义不只是“看数字”, 而是确认硬件拓扑是否合理、collective 是否真正走到了高速链路、以及训练瓶颈究竟在计算还是在通信。

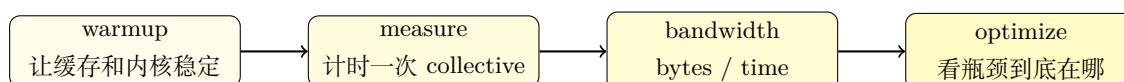


图 8: 带宽测试的工程闭环: 先测, 再算, 再优化

为什么要 benchmark

分布式训练里, 理想带宽和实际带宽通常不是一回事。拓扑、张量大小、链路拥塞、协议选择都会影响结果。你不测, 很多结论都只是猜。

3.5 通信复杂度怎么估

讲里反复强调的一点是: collective 的性能不是只看“有没有通信”, 而是看 **通信次数、每次传多少、以及能不能和计算重叠**。一个常见的一阶模型是

$$T_{\text{comm}}(S, p) \approx \alpha \cdot N_{\text{msg}} + \frac{S}{B_{\text{eff}}}$$

其中 S 是总数据量, p 是 rank 数, α 表示单次消息启动开销, B_{eff} 是有效带宽。

对 ring all-reduce 来说，一个更具体的近似是

$$T_{\text{all-reduce}}(S, p) \approx 2(p-1)\alpha + 2 \frac{p-1}{p} \frac{S}{B_{\text{eff}}}.$$

这个式子想表达的不是“精确计时”，而是两个事实：**rank 越多，启动次数越多；张量越大，带宽项越重要。**

项	含义	工程直觉
α	启动一次通信的代价	小张量时它会主导总时间
S/B_{eff}	真正搬运数据的时间	大张量时它决定下限
$2(p-1)\alpha$	ring 的多步调度开销	rank 越多，控制面越重
$2 \frac{p-1}{p} S$	每个 rank 需要传输的总字节数	比 naive 广播更省带宽

表 5: all-reduce 复杂度里的两个核心项

一个 4 卡的量化例子

假设需要同步一个 64 MiB 的梯度张量， $p = 4$ ，有效带宽按 50 GiB/s 估计，启动开销按 $5 \mu s$ 估计。则

$$T \approx 6\alpha + 1.5 \times \frac{64}{50} \text{ ms} \approx 30 \mu s + 1.92 \text{ ms}.$$

也就是说，哪怕控制面几乎可以忽略，光带宽项也已经接近 2ms。若单步反向计算本身只有一两毫秒，通信就会立刻变成主瓶颈。

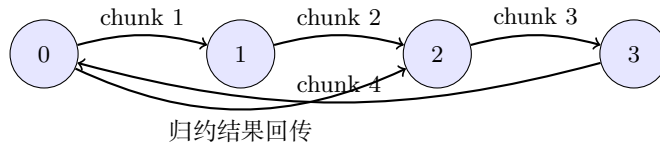


图 9: ring all-reduce 的直觉：数据沿着环走，归约在传递过程中逐步完成

为什么小张量常常跑不满

启动一次 collective 的固定开销并不会因为张量变小而消失。张量越小，有效带宽越容易被启动延迟和同步等待吞掉，所以小张量经常看起来“理论上该快，实际上不快”。

3.6 all-reduce 和 reduce-scatter 的关系

把 all-reduce 拆成 reduce-scatter 和 all-gather，不只是教科书上的形式变换，而是底层实现优化的重要入口。前者把“每个人都拥有完整结果”先缩成“每个人先持有一段结果”，后者再把分段结果拼回去。这样做的好处是，通信模式更接近硬件可以并行执行的方式。

Ring all-reduce 的带宽最优性

Ring all-reduce 的核心性质是：无论有多少个 rank，每个 rank 的总通信量都是 $2 \cdot \frac{p-1}{p} \cdot S$ ，其中 S 是要同步的张量大小。这意味着：

- 当 $p \rightarrow \infty$ 时，每 rank 通信量趋近 $2S$ ，**不随 rank 数线性增长**。
- 这是带宽最优的：理论下界要求每个 rank 至少发送 $\frac{p-1}{p}S$ 和接收 $\frac{p-1}{p}S$ ，ring 实现恰好达到此下界。
- 拆分为两步后，reduce-scatter 阶段传输 $\frac{p-1}{p}S$ ，all-gather 阶段再传输 $\frac{p-1}{p}S$ ，总和恰为 $2 \cdot \frac{p-1}{p}S$ 。

这也解释了为什么 NCCL 默认采用 ring-based 算法：它在大张量场景下几乎达到了链路的理论吞吐上限。

Ring all-reduce 的具体执行分为两个阶段：

1. **Reduce-scatter 阶段** ($p-1$ 步)：每个 rank 把张量分成 p 个 chunk；在每一步中，每个 rank 向环中的下一个 rank 发送一个 chunk，同时接收上一个 rank 的 chunk 并做归约。 $p-1$ 步后，每个 rank 持有完整归约结果的 $\frac{1}{p}$ 。
2. **All-gather 阶段** ($p-1$ 步)：每个 rank 把自己持有的归约完成的 chunk 沿环传递。 $p-1$ 步后，所有 rank 都拥有完整的归约结果。

实现方式	通信模式	常见代价
直接 all-reduce	逻辑最简单	有时不如分解后容易 overlap
reduce-scatter + all-gather	先分片再拼回	更适合 pipeline 化实现
树形归约	归约深度较小	依赖拓扑和实现细节

表 6: 同一个语义，不同实现路径的工程取舍

3.7 怎么读一个 all-reduce 的数字

all-reduce 不是简单的“传一次大包”，而是把 **归约**和 **广播**合在一起。对于梯度同步来说，它的语义是“每个 rank 都贡献一份梯度，然后大家最终都拿到同一个平均值”。工程上常见的实现会拆成 reduce-scatter 和 all-gather 两步，目的不是换个名字，而是让通信更贴近硬件可以并行的方式。

带宽数字背后的真正问题

如果测出来的数值很低，问题未必在算法本身，可能是：

- 张量太小，启动开销占比过高；
- 链路没走到预期拓扑；
- 某个 rank 提前阻塞，导致整体被拖慢；
- 计算和通信没有 overlap。

所以 benchmark 的意义，不是单纯做报表，而是把“慢在哪里”缩小到能解释的粒度。

4 数据并行：按 batch 切分

数据并行是最直接的并行方式：**每个 rank 都持有完整模型，但只处理一部分 batch**。前向和反向都在本地完成，然后在反向结束后对梯度做 all-reduce，同步所有 worker 的参数更新。

4.1 工作方式

1. 将 batch 沿 batch 维度切成多份。
2. 每个 rank 拿到自己的本地数据片段。
3. 每个 rank 维护一份完整模型副本。
4. 本地完成前向、损失和反向。
5. 对每层 `param.grad` 做 all-reduce 平均。
6. 各自执行 optimizer step。

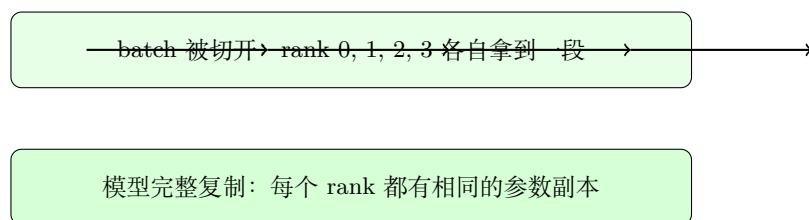


图 10: 数据并行的切分方式：数据分片，模型复制

DDP 的本质

数据并行里，**参数是复制的，数据是切分的，梯度是同步的**。从优化器视角看，像是每个 rank 都在跑 SGD，但梯度会在每步被强制对齐。

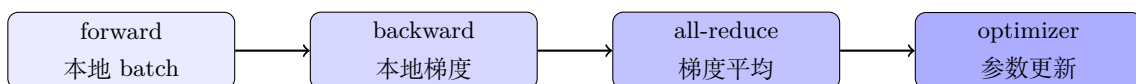


图 11: DDP 的关键步骤：本地前向反向之后，只同步梯度

本地 loss 可能不同
因为看见的 batch 不同

all-reduce 后梯度一致
参数更新就一致

不同 rank 的模型副本
会继续保持同形

图 12: 为什么 loss 可以不同, 但参数仍一致

4.2 DDP 的代价与边界

DDP 的优点是实现简单、计算清晰、适用面广。缺点也很明显：**模型和优化器状态都要完整复制**到每个 GPU 上。当模型太大时，单纯的数据并行就不够用了。

优点	缺点	适用场景
实现最简单，语义最直观	参数和 optimizer state 全复制	模型能放进单卡，但 batch 很大
梯度同步只发生一次/step	模型越大越吃显存	先从 DDP 开始最稳妥
工程生态成熟	不解决超大模型的单卡存储问题	训练任务比较常规时

表 7: 数据并行的工程权衡

一个需要注意的边界条件

如果模型里有依赖整个 batch 统计量的层，比如某些 batch norm 变体，就需要额外考虑跨 rank 的统计同步。讲这门课主要用的是 MLP 和 transformer 语境，所以经常更接近 layer norm，但工程里这个细节仍然要盯住。

4.3 为什么 DDP 不是“把单卡训练复制四份”就结束

表面上看，DDP 像是每张卡都在独立训练，然后最后把梯度平均一下。但真正的代价还在 **优化器状态**和 **参数副本**上。以 Adam 为例，除了参数本身，你还要保存一阶、二阶动量；模型一大，这部分显存开销就会很快吃掉空间。

DDP 的隐含成本

DDP 省的是“实现复杂度”，不是“模型总内存”。如果参数、梯度、优化器状态加起来已经逼近单卡上限，DDP 只能帮你并行算，不会帮你凭空消失这些状态。

4.4 内存账本：DDP 到底在占什么

如果只看参数大小，DDP 似乎很轻松；真正把显存吃掉的，通常是 **梯度**、**优化器状态**和 **激活**。对于一个参数总量为 P 的模型，在混合精度下一个常见的粗略账本是：

$$M_{\text{DDP}} \approx M_{\text{params}} + M_{\text{grads}} + M_{\text{master}} + M_{\text{Adam}} + M_{\text{act}}.$$

如果参数和梯度用 fp16，各占 $2P$ bytes；master 权重用 fp32，占 $4P$ bytes；Adam 的一阶、二阶动量各用 fp32，占 $8P$ bytes，那么仅静态训练状态就已经接近

$$16P \text{ bytes / parameter.}$$

这也是为什么“模型参数看起来不算离谱”，但一跑训练就爆显存的情况非常常见。

状态	每参数占用	谁最容易忽略
fp16 参数	2 bytes	初学者常只看这个
fp16 梯度	2 bytes	反向之后会短暂同时存在
fp32 master weight	4 bytes	混合精度训练常见
Adam m / v	8 bytes	真正的大头之一
activation	依赖 batch 和层数	通常随 microbatch 变化

表 8: 训练显存的常见账本

Adam m/v	最容易吃满显存
master	混合精度常驻
grads	反向期短暂存在
params	复制到每个 rank

图 13: DDP 的静态显存构成：看起来是“复制模型”，实际上复制的是整套训练状态

为什么激活也不能忘

激活的大小通常跟 local batch、sequence length、隐藏宽度成正比。DDP 只是在数据维度上切开 batch，但如果 local batch 仍然不小，激活一样会把显存顶满。

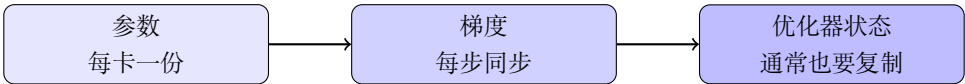


图 14: DDP 不只是同步梯度，背后还要承担参数和优化器状态的常驻内存

为什么数据并行仍然是默认起点

因为它最接近“把单卡训练复制 N 份再同步”的直觉，最容易调试，也最容易验证正确性。很多复杂并行策略最终仍会回到数据并行作为外层骨架。

```
1 x = local_batch
2 for W in params:
3     x = gelu(x @ W)
4 loss = x.square().mean()
5 loss.backward()
6 for W in params:
7     dist.all_reduce(W.grad, op=AVG)
8 optimizer.step()
```

Listing 1: 数据并行的极简理解

4.5 一个 DDP 的数值化判断

假设模型总参数量是 200 M，那么仅静态训练状态大约就是

$$200 \text{ M} \times 16 \text{ bytes} \approx 3.2 \text{ GiB}.$$

如果再加上中间激活、通信缓冲区和框架开销，单卡显存很容易被迅速吃满。这个算式的意义不是精确到最后一位，而是帮助你判断：**当模型已经接近单卡极限时，DDP 只是把问题复制了多份，并没有把问题消掉。**

DDP 的一个常见误判

很多人会把”每卡只看 local batch”误解成”显存会很省”。实际上 local batch 小了以后，激活会下降，但参数、梯度和 Adam 状态没有因此变少，所以 DDP 的总显存仍然可能很高。

4.6 Worked Example: Llama 2 13B 的 DDP 通信开销

为了建立对真实模型通信代价的直觉，我们用 Llama 2 13B 作为算例：

参数	数值	说明
总参数量	13B	130 亿参数
参数精度	fp16 (2 bytes)	混合精度训练
梯度大小	$13\text{B} \times 2 = 26 \text{ GB}$	每个参数一个梯度
GPU 数	4	同一节点，NVLink 连接
互联带宽	450 GB/s (有效)	NVLink 4.0 单向有效带宽

表 9: Llama 2 13B DDP 通信开销计算的基本参数

每步训练需要做一次梯度 all-reduce。Ring all-reduce 下，每个 rank 的通信量为：

$$V_{\text{comm}} = 2 \cdot \frac{p-1}{p} \cdot S = 2 \times \frac{3}{4} \times 26 \text{ GB} = 39 \text{ GB}$$

通信耗时估算：

$$T_{\text{comm}} = \frac{V_{\text{comm}}}{B_{\text{eff}}} = \frac{39 \text{ GB}}{450 \text{ GB/s}} \approx 87 \text{ ms}$$

项目	估算值	占比分析
单步前向 + 反向 (A100)	~200–400 ms	计算主体
梯度 all-reduce (NVLink)	~87 ms	通信可与反向重叠
梯度 all-reduce (InfiniBand)	~975 ms	跨节点会严重拖慢训练
梯度 all-reduce (100G Ethernet)	~3900 ms	基本不可用

表 10: Llama 2 13B 在不同互联下的通信开销对比

DDP 的通信可以与计算重叠

PyTorch DDP 的一个关键优化是：不必等所有层的反向都结束再做 all-reduce。当最后一层的梯度算完，就可以立即开始该层的 all-reduce，同时继续计算倒数第二层的梯度。这种 **bucket gradient all-reduce** 让通信和计算在时间上重叠，实际的额外延迟远小于 87 ms。但这要求通信带宽足够高——如果通信比计算还慢，重叠也无法完全隐藏延迟。

DDP 的显存账单

Llama 2 13B 在 DDP 下，每张卡需要存储：

- fp16 参数： $13\text{B} \times 2 = 26\text{ GB}$
- fp16 梯度：26 GB
- fp32 master 权重： $13\text{B} \times 4 = 52\text{ GB}$
- Adam 一阶/二阶动量： $13\text{B} \times 8 = 104\text{ GB}$

仅静态状态就需要 $\sim 208\text{ GB}$ ，远超单张 A100 (80 GB) 或 H100 (80 GB) 的显存。这就是为什么 13B 及以上模型几乎不可能用纯 DDP 训练——必须引入 ZeRO 或模型并行。

5 张量并行：按宽度切分

数据并行解决的是“模型放得下吗”，张量并行解决的是更难的问题：**如果单层矩阵都太大，怎么把一层内部再切开**。它的思路是切 hidden dimension，让每个 rank 只持有一部分参数和一部分中间激活。

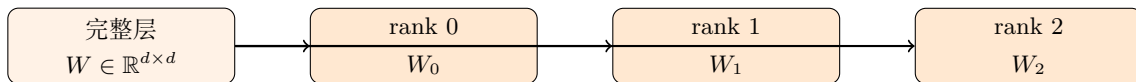


图 15: 张量并行的核心：把一层里的宽度切成多份

5.1 工作方式

在 lecture 的简化 MLP 里，每层都是矩阵乘法加非线性。张量并行的做法是：每个 rank 只算自己的参数切片，算完之后再吧激活拼回完整形状。于是，**参数省了，通信却变多了**。

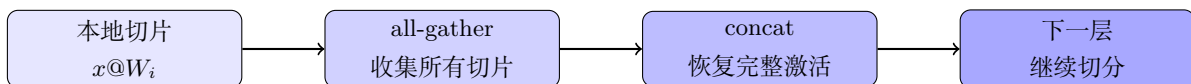


图 16: 张量并行的前向过程：每层都需要把部分结果重新拼回去

为什么张量并行要求更快的互联

因为它往往是“每层都要通信”。如果互联慢，模型虽然被切开了，但中间激活会反复来回搬，最后把计算收益全吃掉。

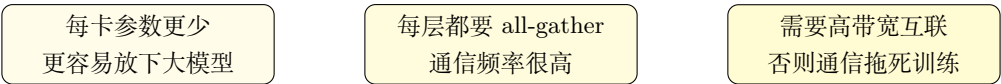


图 17: 张量并行的三角权衡：更省显存，但更吃互联

张量并行的脆弱点

如果 interconnect 不够快，张量并行会比数据并行更容易被通信拖垮。它适合”单层就已经很大”的模型，但不适合拿来粗暴替代所有其他并行策略。

张量并行要求低延迟互联——只适合 NVLink 范围

张量并行的通信发生在**每一层**的前向和反向中，而不是像 DDP 那样每步只做一次。对于一个 40 层的 transformer，张量并行意味着每步至少 $40 \times 2 = 80$ 次 all-gather 或 reduce-scatter（前向 + 反向各一次）。如果互联延迟从 NVLink 的 $\sim 1 \mu s$ 变为 InfiniBand 的 $\sim 5 \mu s$ ，80 次调用的累积延迟从 $80 \mu s$ 变为 $400 \mu s$ ——看起来不大，但加上带宽瓶颈，影响会成倍放大。

实践准则：张量并行只在 NVLink 可达的 GPU 之间使用（通常是同一节点的 8 张卡）。跨节点的并行需求应由数据并行或流水线并行承担。

5.2 张量并行 vs 数据并行：通信量对比

把 TP 和 DP 的通信开销放在同一个坐标系里看，能更直观地理解两者的权衡。假设模型有 L 层，每层参数大小为 W ，激活大小为 A ：

维度	数据并行 (DDP)	张量并行 (TP)	比较
通信频率	每步 1 次	每层 2 次（前向 + 反向）	TP 频率高 $\sim 2L$ 倍
每次通信量	$2 \cdot \frac{p-1}{p} \cdot LW$	$2 \cdot \frac{p-1}{p} \cdot A$	取决于 LW vs A
总通信量/步	$2 \cdot \frac{p-1}{p} \cdot LW$	$2L \cdot 2 \cdot \frac{p-1}{p} \cdot A$	DP 通信与参数成正比
通信类型	all-reduce（梯度）	all-gather + reduce-scatter	TP 更敏感于延迟
互联要求	InfiniBand 可接受	必须 NVLink	TP 对硬件更挑剔

表 11: 数据并行 vs 张量并行通信量对比

一个具体的数字对比

以 Llama 2 13B（40 层， $d = 5120$ ， $B = 128$ ， $p = 4$ ，fp16）为例：

- **DDP 每步总通信量：** $2 \times \frac{3}{4} \times 26 \text{ GB} = 39 \text{ GB}$ ，但只需 1 次 all-reduce。
- **TP 每步总通信量：**每层前向 all-gather 激活 $\approx 128 \times 5120 \times 2 = 1.25 \text{ MB}$ ，40 层前向 + 反向共 $\approx 80 \times 1.25 \text{ MB} \times \frac{3}{4} \times 2 \approx 150 \text{ MB}$ 。
- TP 的总字节数远小于 DDP，但它需要 80 次独立的集合操作，每次都有启动延迟。在 NVLink 上这不是问题，在跨节点互联上就会成为致命瓶颈。

5.3 为什么张量并行的反向传播更难

前向里，我们还能把一层的输出拆开并在必要时拼回去；但在反向里，梯度流向会反过来，意味着你常常既要知道“每个 shard 自己贡献了什么”，也要知道“所有 shard 一起到底形成了什么”。这就是为什么张量并行经常会配合 `all_gather`、`reduce_scatter` 或者等价的分片同步原语。

张量并行的关键不是切片，而是配套的梯度路径

只把矩阵切开还不够。前向、反向、参数更新必须共享同一套分片约定，否则每个 rank 看到的只是局部真相，拼不回正确的整体梯度。

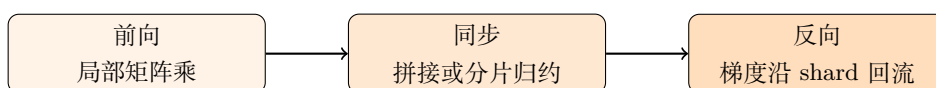


图 18: 张量并行的前后向必须配套设计，否则局部计算无法恢复全局正确性

```
1 x = full_batch
2 for W_local in shard_of_each_layer:
3     y_local = gelu(x @ W_local)
4     all_parts = all_gather(y_local)
5     x = cat(all_parts, dim=-1)
```

Listing 2: 张量并行的极简理解

张量并行的核心直觉

它不是把模型“切碎”就结束，而是每一层之后都要恢复形状。换句话说，张量并行把压力从“显存”转移到了“通信链路”。

5.4 一个张量并行的数值化例子

把上面的直觉换成数字会更清楚。假设输入激活大小是 $B \times d$ ，其中 batch $B = 128$ ，hidden width $d = 1024$ ，采用 fp16，那么单个激活张量的大小大约是

$$128 \times 1024 \times 2 \text{ bytes} = 256 \text{ KiB}.$$

如果把 hidden width 切成 4 份，那么每个 rank 先算自己的 $\frac{1}{4}$ 激活，再通过 all-gather 恢复完整张量。按 ring 近似，每个 rank 一层的通信量大约是

$$2^{\frac{p-1}{p}} A = 1.5A \approx 384 \text{ KiB},$$

其中 A 是完整激活大小。这个数看上去不大，但注意它是 **每层都要发生**。

量	数值例子	解释
batch B	128	讲里常见的小批量起点
hidden d	1024	一个不算夸张的层宽
激活大小 A	256 KiB	单层完整输出
4 卡 all-gather 代价	384 KiB / layer / rank	频繁发生，所以要快链路

表 12: 张量并行的数字直觉

为什么张量并行更像“层内系统优化”

因为它让通信频率提高到“按层计”，于是单层算得再快，也必须保证通信不会把收益抵消。这和 DDP “按步同步”的节奏完全不同。

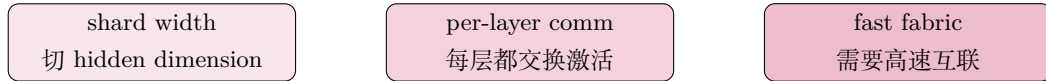


图 19: 张量并行的三步逻辑：切宽度、传激活、靠高速互联兜底

6 流水线并行：按深度切分

流水线并行的思路是：**模型太深时，把层切到不同 rank 上**。这样每个 rank 只负责一段层堆栈，前一个 rank 算完就把激活传给下一个 rank。

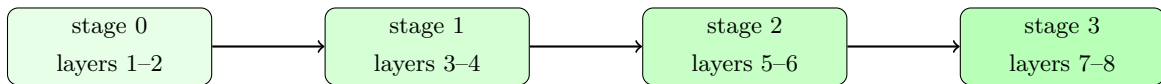


图 20: 流水线并行的切分方式：层分到不同 stage，激活在 stage 之间流动

6.1 为什么要 microbatch

如果一个大 batch 整体往前推进，后面的 stage 会空等前面的 stage。这种空闲就是 pipeline bubble。解决方法是把 batch 切成 microbatches，让不同 stage 同时处理不同 microbatch，从而提高利用率。

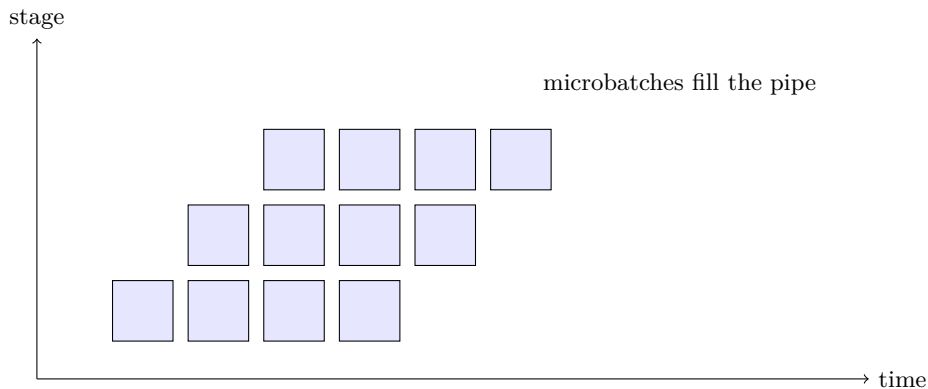


图 21: microbatch 让流水线不再空转

bubble 的直觉

stage 越多，启动和收尾的空档越明显。microbatch 越少，空档越浪费；microbatch 越多，利用率越高，但调度也更复杂。

6.2 流水线为什么总有空档

流水线并行的难点不是“能不能传过去”，而是“什么时候能持续传”。当最前面的 stage 刚开始工作时，后面的 stage 还没拿到激活；当最后的 stage 已经快结束时，前面的 stage 也没法继续喂

新数据。这种首尾空档是流水线天然会有的。

microbatch 的作用

microbatch 本质上是在增加流水线里的“在途任务数”。在途任务越多，stage 的空转越少；但 microbatch 太碎又会增加调度次数和通信次数，所以这里也不是越多越好。

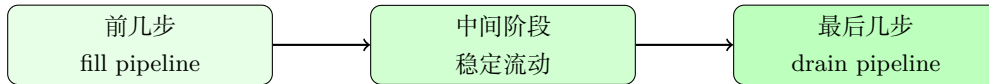


图 22: 流水线的三个时期：灌入、稳定、排空

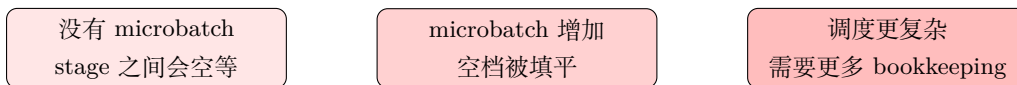


图 23: pipeline bubble 的本质：算得太慢不是唯一问题，等得太久也会浪费

6.3 点对点通信

流水线并行和前两种策略的一个重要差别是，它更依赖 `send / recv` 这类点对点通信。rank 之间只传激活，而不是每步都做全局归约。

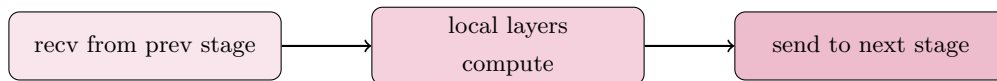


图 24: 流水线并行的 rank 内循环：先接收，再计算，再发送

流水线并行的常见问题

- stage 负载不均会让最慢的 stage 决定整体速度。
- microbatch 太少，bubble 很大。
- 计算和通信若不能 overlap，效率会明显掉下来。

```
1 for microbatch in chunks(batch):
2     if rank > 0:
3         recv(x)
4     for W in local_layers:
5         x = gelu(x @ W)
6     if rank + 1 < world_size:
7         send(x)
```

Listing 3: 流水线并行的极简理解

流水线并行的核心

它把“深模型的一串层”拆成几个 stage，然后让激活像传送带一样往前走。难点不在层本身，而在怎么把 stage 排得尽量均衡。

6.4 流水线效率的近似公式

流水线并行不是“把层切开”就自动高效。它的效率常常可以用一个非常粗的近似来理解：

$$\eta_{\text{pipe}} \approx \frac{M}{M + S - 1},$$

其中 M 是 microbatch 数, S 是 stage 数。这个式子直接告诉你: **stage 越多, 空档越大; microbatch 越少, 利用率越低。**

stage 数 S	microbatch 数 M	近似效率	直觉
4	4	$4/7 \approx 57\%$	空档明显
4	8	$8/11 \approx 73\%$	已开始像流水线
4	16	$16/19 \approx 84\%$	利用率较高
8	8	$8/15 \approx 53\%$	stage 多时更容易浪费

表 13: microbatch 数与流水线效率的关系

microbatch 不是越碎越好

microbatch 越多, pipeline bubble 越小, 但调度、通信和 kernel launch 次数都会上升。实际训练里要在” 利用率” 和” 额外开销” 之间找平衡。

6.5 Worked Example: 流水线 bubble 开销的精确计算

用一组具体数字来感受 bubble 的代价。假设每个 microbatch 通过一个 stage 的计算时间为 t_f (前向) 和 t_b (反向), 且 $t_b \approx 2t_f$ (反向通常比前向慢 ~ 2 倍)。

配置	bubble 时间	有效计算时间	bubble 占比
$S = 4, M = 4, t_f = 10\text{ms}$	$(S-1) \cdot t_f = 30\text{ms}$	$M \cdot (t_f + t_b) = 120\text{ms}$	$30/150 = 20\%$
$S = 4, M = 8$	30ms	240ms	$30/270 = 11\%$
$S = 4, M = 16$	30ms	480ms	$30/510 = 5.9\%$
$S = 4, M = 32$	30ms	960ms	$30/990 = 3.0\%$
$S = 8, M = 8$	70ms	240ms	$70/310 = 22.6\%$
$S = 8, M = 32$	70ms	960ms	$70/1030 = 6.8\%$

表 14: 不同 stage 数和 microbatch 数下的 bubble 开销。 $t_f = 10 \text{ ms}$, $t_b = 20 \text{ ms}$ 。

更一般的 bubble 占比公式为：

$$\text{bubble ratio} = \frac{(S-1) \cdot t_f}{M \cdot (t_f + t_b) + (S-1) \cdot t_f} \approx \frac{S-1}{3M + S - 1}$$

其中假设 $t_b \approx 2t_f$ 。要把 bubble 控制在 5% 以内, 需要 $M \geq \frac{19(S-1)}{3}$ 。对 $S = 4$, 这意味着 $M \geq 19$; 对 $S = 8$, 需要 $M \geq 44$ 。

流水线并行的 Bubble 问题与 1F1B 调度

上面的分析假设的是最朴素的 GPipe 调度——先做完所有 microbatch 的前向，再做所有 microbatch 的反向。这种调度有两个问题：

1. **Bubble 大**：fill 和 drain 阶段都有空闲 stage。
2. **激活内存高**：所有 microbatch 的中间激活必须同时保留，直到反向计算。

1F1B (One Forward One Backward) 调度通过交替执行前向和反向来缓解这两个问题：

- 在 steady state 阶段，每个 stage 交替做一次前向、一次反向。
- 激活内存从 $O(M)$ 降低到 $O(S)$ ——每个 stage 只需保留 S 个 microbatch 的激活。
- Bubble 比例与 GPipe 相同 ($\frac{S-1}{M+S-1}$)，但内存效率大幅提升。

更进一步的调度如 **interleaved 1F1B** (Megatron-LM 使用) 可以把 bubble 再减半，代价是每个 rank 需要持有多段不连续的层。

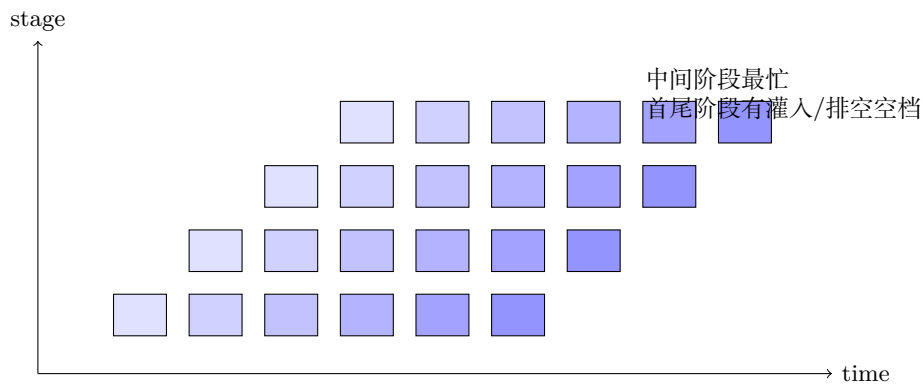


图 25: 流水线调度的典型形态：不是每个 stage 都能同时满负载

6.6 stage 负载不均会怎样

如果某个 stage 远重于其他 stage，那么整条 pipeline 都会被拖慢。换句话说，流水线的吞吐量大致由最慢的 stage 决定，而不是由平均 stage 决定。这也是为什么切 stage 时不能只按层数平均，还要看每层到底有多少计算量。

流水线切分的核心准则

切 stage 时，不是“每个 rank 分到一样多的层”就够了，而是要尽量让每个 stage 的计算时间接近。否则瓶颈 stage 会把整条流水线的效率压低。

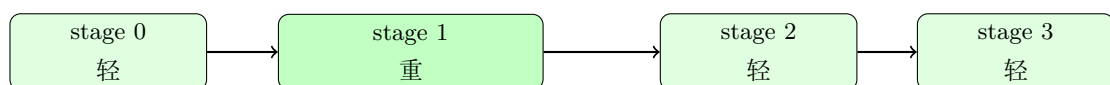


图 26: stage 不均衡会把吞吐量拉到最重的那一段上

7 如何选择并行策略

7.1 三种策略的直观对比

现在可以把三种并行方式放在一起看：

策略	切分对象	主要通信	主要优点	主要缺点
数据并行	batch	每步梯度 all-reduce	最简单、最稳	模型要能单卡放下
张量并行	hidden width	每层 all-gather / reduce-scatter	能切更大的层	互联要求高
流水线并行	depth / layers	stage 间 send / recv	深模型更自然	bubble 与调度复杂

表 15: 三种并行方式的核心差别

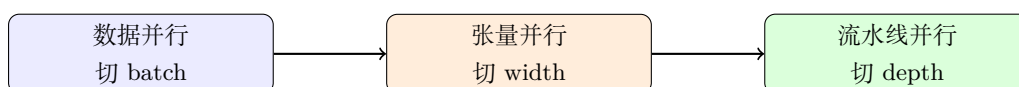


图 27: 三种并行策略分别对应 batch、width、depth 三个维度

为什么经常要组合使用

单一策略很少同时解决“显存不够、单层太大、总层数太深”这三个问题。现实训练通常会把数据并行放在外层，再叠加张量并行或流水线并行。

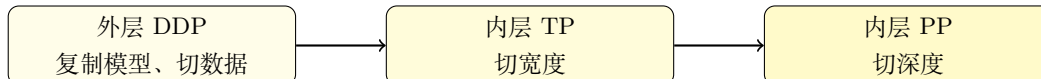


图 28: 常见的组合思路：外层数据并行，内层张量并行或流水线并行

没有免费午餐

切得越细，单卡显存压力越小，但通信和调度复杂度越高。分布式训练不是“让模型免费变大”，而是“把复杂度从显存转移到系统设计上”。

7.2 三种并行策略的详细对比

下面这张表从更多维度对比三种核心并行策略，帮助建立完整的决策框架：

维度	数据并行	张量并行	流水线并行
切分对象	batch 维度	hidden width 维度	depth (层) 维度
每卡存储	完整模型 + 优化器	$\frac{1}{p}$ 的参数	$\frac{1}{p}$ 的层
通信原语	all-reduce	all-gather, reduce-scatter	send / recv
通信频率	每步 1 次	每层 2 次	每 stage 边界 1 次
通信量/步	$O(\text{params})$	$O(L \times \text{activation})$	$O(\text{activation})$
互联要求	InfiniBand 够用	必须 NVLink	中等 (点对点)
典型 GPU 范围	跨节点可用	同节点 2-8 卡	跨节点可用
实现复杂度	低 (DDP 一行代码)	中高	高 (调度复杂)
主要开销	显存冗余	通信延迟	pipeline bubble
扩展性瓶颈	模型必须放进单卡	互联带宽	stage 负载均衡

表 16: 三种并行策略的多维度对比。 L 为层数, p 为并行度。

7.3 一个实用的决策表

如果你遇到...	优先考虑	原因
模型能放下, 但 batch 很大	数据并行	最少改动、最容易扩展
单层矩阵已经太大	张量并行	先把层宽切开
模型很深, stage 很重	流水线并行	让层分布到不同设备
三者都很紧张	组合方案	现实训练通常只能组合解决

表 17: 选择并行策略的实用起点

工程上最重要的一句

并行策略不是先选“最酷”的, 而是先看瓶颈到底在哪里: 是模型太大、单层太宽, 还是通信太慢。瓶颈不同, 切法就不同。

从教学角度看 MLP 的价值

讲里用深 MLP 作为例子, 不是因为 MLP 最重要, 而是因为它最容易看清楚本质: 参数如何复制、数据如何切、激活如何走、梯度如何同步、通信在哪里发生。把这套逻辑看懂, 后面再看 transformer 也会更清楚。

8 一个四卡算例：同一模型的三种切法

为了把前面的公式落到实处，考虑一个玩具但够用的例子：一个 12 层的 MLP，隐藏宽度 $d = 4096$ ，batch size $B = 128$ ，使用 fp16 训练，设备数 $p = 4$ 。这个例子不是为了模仿某个真实模型，而是为了把 DDP、TP、PP 的代价放在同一个坐标系里。

设定	数值	说明
层数	12	便于做 pipeline 划分
隐藏宽度	4096	足够大，能看出通信成本
batch size	128	不算极端的大 batch
GPU 数	4	便于直接比较三种切法

表 18: 四卡算例的基本设定

在这个设定下，单层权重矩阵大约有 $4096 \times 4096 \approx 16.8\text{M}$ 参数，也就是 fp16 下约 32 MiB。12 层总参数量接近 201M，静态训练状态在 DDP 里大约是

$$201\text{M} \times 16 \text{ bytes} \approx 3.2 \text{ GiB},$$

还没算激活和缓冲区。另一方面，一个 batch 的激活大小约为

$$128 \times 4096 \times 2 \text{ bytes} = 1 \text{ MiB}.$$

这意味着：DDP 的主要痛点是状态复制，TP 的主要痛点是按层通信，PP 的主要痛点是 bubble。

方案	单卡持有内容	每步/每层通信	第一感觉	典型风险
DDP	完整模型 + 完整 optimizer state	每步一次梯度 all-reduce	最容易跑通	状态太大，显存压力高
TP	约 $\frac{1}{4}$ 的宽度切片	每层 all-gather / reduce-scatter	模型能切更细	互联带宽不够会拖死
PP	约 $\frac{1}{4}$ 的层堆栈	stage 间 send / recv	stage 化更自然	bubble 和负载不均

表 19: 同一个模型在三种切法下的直观代价

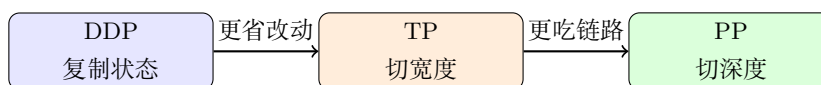


图 29: 同一模型在不同切分维度上的权衡

这个算例想让你看到什么

同一个模型，换一种切法，瓶颈位置就完全变了。DDP 的关键是状态大小，TP 的关键是每层通信，PP 的关键是调度空档。并行策略不是“哪个更高级”，而是“哪个瓶颈更像你的真实约束”。

9 实际系统中的并行策略

9.1 ZeRO: 在数据并行中消除冗余

DDP 的最大问题是每张卡都完整复制了参数、梯度和优化器状态。微软的 **ZeRO** (Zero Redundancy Optimizer) 通过分片来消除这种冗余，分为三个递进的阶段：

ZeRO 的三个阶段

- **ZeRO-1** (优化器状态分片): 每个 rank 只保存 $\frac{1}{p}$ 的 Adam 状态 (m 和 v)。训练步骤中, 每个 rank 只更新自己负责的那部分参数, 然后通过 all-gather 广播更新后的参数。**显存节省**: 以 13B 模型为例, Adam 状态从 104 GB/卡降到 26 GB/卡 (4 卡)。
- **ZeRO-2** (+ 梯度分片): 在 ZeRO-1 基础上, 梯度也只保留 $\frac{1}{p}$ 。反向传播时用 reduce-scatter 代替 all-reduce——每个 rank 只保留自己负责参数的归约梯度。**额外节省**: 梯度从 26 GB/卡降到 6.5 GB/卡。
- **ZeRO-3** (+ 参数分片): 连参数本身也分片。前向和反向需要完整参数时, 通过 all-gather 临时收集, 用完立即释放。**极致节省**: 每卡只需 $\frac{1}{p}$ 的全部训练状态。

阶段	分片内容	每卡显存	额外通信
DDP (基线)	无分片	$16P$ bytes	1 次 all-reduce
ZeRO-1	优化器状态	$4P + \frac{12P}{p}$ bytes	1 次 all-gather (参数)
ZeRO-2	+ 梯度	$2P + \frac{14P}{p}$ bytes	reduce-scatter + all-gather
ZeRO-3	+ 参数	$\frac{16P}{p}$ bytes	前向/反向各 1 次 all-gather

表 20: ZeRO 各阶段的显存与通信权衡。 P 为参数量, p 为 GPU 数。

ZeRO-3 的隐含代价

ZeRO-3 虽然把显存降到最低, 但它要求在前向和反向的每一层都做 all-gather 来收集完整参数——这和张量并行的通信频率类似。如果跨节点使用 ZeRO-3, 通信开销可能反而比 ZeRO-2 + 流水线并行更大。实践中, ZeRO-2 通常是性价比最高的选择。

9.2 实际训练框架的并行策略对比

维度	Megatron-LM	DeepSpeed ZeRO	PyTorch FSDP
核心思路	3D 并行	数据并行 + 状态分片	类 ZeRO-3 的参数分片
张量并行	原生支持	需配合 Megatron	不直接支持
流水线并行	原生 (1F1B)	ZeRO + PP	不直接支持
数据并行	原生	ZeRO-1/2/3	原生 (分片式)
典型规模	100B+ 参数	10B-1T 参数	1B-100B 参数
代码侵入性	高 (需改模型)	中 (包装优化器)	中 (包装模块)
适用场景	超大规模预训练	灵活部署	PyTorch 生态内

表 21: 主流分布式训练框架对比

Megatron-LM 的 3D 并行

Megatron-LM 是 NVIDIA 开发的大模型训练框架，它同时使用三种并行：

- **张量并行 (TP)**：在同一节点的 8 张 NVLink 连接的 GPU 间切分层宽。
- **流水线并行 (PP)**：跨节点切分层深度，用 interleaved 1F1B 调度。
- **数据并行 (DP)**：在 TP×PP 组之间复制，用 all-reduce 同步梯度。

例如训练 GPT-3 175B 时，一种典型配置是 TP=8 (节点内)，PP=8 (跨 8 个节点)，DP=16 (16 组副本)，共需 $8 \times 8 \times 16 = 1024$ 张 GPU。

10 全讲总结

这一讲其实只想回答一个问题：当模型规模和硬件规模一起变大时，训练系统应该怎样分配计算与通信？

10.1 本章小结

三条最终 takeaway

1. **数据并行**解决的是 batch 维度扩展，最简单也最常用。
2. **张量并行**解决的是单层太宽的问题，但它非常吃互联带宽。
3. **流水线并行**解决的是模型太深的问题，但它要面对 bubble 和调度复杂度。

核心概念	一句话解释	关键公式	工程要点
集合通信	多 rank 间的标准协议	$T = \alpha + S/B$	NCCL 自动优化路径
Ring all-reduce	带宽最优的全局归约	$2 \frac{p-1}{p} S$	大张量时接近理论峰值
数据并行	复制模型，切分 batch	每步 all-reduce $O(P)$	先从 DDP 开始
张量并行	切分层宽度	每层 all-gather $O(A)$	仅限 NVLink 范围
流水线并行	切分层深度	bubble $\frac{S-1}{M+S-1}$	microbatch 数要够大
ZeRO	分片训练状态	每卡 $\frac{16P}{p}$	ZeRO-2 性价比最高

表 22: 全讲核心概念速查表

10.2 这节课没有覆盖的内容

讲者在最后提到了几个本讲没有深入的主题：

- **更通用的模型**：本讲用 MLP 演示，真实 transformer 还有 attention 层需要特殊的分片策略（如 Megatron 的 column/row parallel）。
- **通信/计算重叠**：DDP 的 bucket gradient all-reduce、1F1B 调度等技术可以让通信隐藏在计算背后，但代码实现复杂度会大幅上升。
- **序列并行**（Sequence Parallelism）：当序列长度非常长时，可以沿 sequence 维度切分——这是除了 batch、width、depth 之外的第四个维度。
- **JAX/TPU 的声明式方法**：在 JAX 生态中，用户只需声明分片策略，编译器自动插入通信原语。PyTorch 则要求手动编排，但能看清底层发生了什么。

最后一句话

如果你记住一件事，那就是：多 GPU 训练的本质不是让 GPU 彼此热闹起来，而是让通信次数、通信量和同步频率，恰好配得上模型规模。

10.3 拓展阅读

- Megatron-LM 论文: *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM* (Narayanan et al., 2021)
- ZeRO 论文: *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models* (Rajbhandari et al., 2020)
- PyTorch FSDP 文档: <https://pytorch.org/docs/stable/fsdp.html>
- NCCL 性能分析: <https://github.com/NVIDIA/nccl-tests/blob/master/doc/PERFORMANCE.md>
- GPipe 论文: *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism* (Huang et al., 2019)
- PipeDream 论文 (1F1B 调度): *PipeDream: Generalized Pipeline Parallelism for DNN Training* (Narayanan et al., 2019)